

Покроковий розбір коду

Детальне пояснення кожного рядка трьох файлів проекту: `sdf_model.py`, `sdf_inference.py` та `train.ipynb`. Цей документ — компаньон до `TUTORIAL.md`, який пояснює теорію. Тут — інженерія.

Автор: Ігор Іванишин Дата: 19 травня 2026

Зміст

- Файл 1: `sdf_model.py` — модель PyTorch для тренування
 - Файл 2: `sdf_inference.py` — швидкий Numba-inference
 - Файл 3: `train.ipynb` — тренувальний пайплайн
-

Файл 1: `sdf_model.py`

Цей файл містить **PyTorch-реалізацію** нейронної SDF-моделі. Використовується для тренування. Не для inference (для inference є окремий швидкий numba-файл — див. далі).

Імпорти (рядки 1–6)

```
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
```

- **`dataclass`** — декоратор зі стандартної бібліотеки Python для автогенерації boilerplate-коду класу (конструктора, `__repr__`, тощо). Використовується для `HashConfig`.
- **`Path`** — об'єктно-орієнтований шлях до файлу замість сирих рядків. Безпечніше і платформно-незалежно.

- `numpy` як `np` — стандартна бібліотека числових операцій. Використовується для попередніх обчислень (резолюції рівнів) і експорту.
- `torch` — PyTorch, основна бібліотека deep learning.
- `torch.nn` як `nn` — модуль для шарів нейронних мереж (`Linear`, `Sequential`, `ReLU` тощо).

Прості числа для хешу (рядок 8)

```
_PRIMES = (1, 2_654_435_761, 805_459_861)
```

- Це **три великі прості числа** з оригінальної статті Instant-NGP.
- Перше — `1` — фактично "не множимо". Це константа, через яку проходить координата i .
- Друге `2_654_435_761` — це $\lfloor 2^{32} \cdot \phi - 2^{32} \rfloor$, де ϕ — золотий перетин. Класичне число для **геш-функції Кнута**, що дає рівномірний розподіл.
- Третє `805_459_861` — також велике взаємно просте.
- Підкреслення `_` між тисячами — це **синтаксичний сахар** Python, який ігнорується інтерпретатором, але робить число читабельним.
- Сама змінна починається з `_`, тобто це "приватна" константа модуля — не призначена для імпорту назовні.

Функція хешування (рядки 11–14)

```
def _spatial_hash(ix: torch.Tensor, iy: torch.Tensor, iz: torch.Tensor, T: int) -> torch.Tensor:
    """Instant-NGP spatial hash. Inputs are int32 voxel-corner coordinates."""
    h = (ix * _PRIMES[0]) ^ (iy * _PRIMES[1]) ^ (iz * _PRIMES[2])
    return h.to(torch.long) % T
```

- **Рядок 11:** Сигнатура функції з **тайп-гінтами** (`: torch.Tensor`). Це не примушує типи (Python не строго типізовано), але допомагає IDE та читачеві.
- **Рядок 13:** Обчислення хешу. Кожна координата множиться на свій простий, потім робиться XOR (`^`). Множення на 1 для `ix` означає, що `x` проходить як є.
- **Рядок 14:** Конвертація в `int64` (бо XOR з великими числами може переповнити `int32`), потім остача від ділення на `T`. Це обмежує індекс до діапазону `[0, T-1]`.

Клас конфігурації (рядки 17–39)

```
@dataclass
class HashConfig:
    n_levels: int = 8
    n_features: int = 2
    log2_table_size: int = 13          # T = 2**13 = 8192
    base_resolution: int = 8          # N_min
    finest_resolution: int = 128      # N_max
    bound: float = 1.0                # domain is [-bound, bound]^3
    hidden: int = 16                  # tiny ReLU layer; 0 = linear decoder
```

- **@dataclass** — декоратор, який автоматично генерує `__init__`, `__repr__`, `__eq__` на основі полів класу.
- **Поля з типами і дефолтними значеннями.** Можна створити `HashConfig()` без аргументів — буде використано всі дефолти.
- Якщо хочемо більший варіант — `HashConfig(log2_table_size=14)` — лише змінюємо одне поле, решта залишаються дефолтними.

```
@property
def T(self) -> int:
    return 1 << self.log2_table_size
```

- **@property** — робить виклик `cfg.T` без дужок, як атрибут.
- **`1 << n`** — побітовий зсув вліво. `1 << 13 = 8192 = 213`. Швидко обчислює степінь двійки.

```
@property
def resolutions(self) -> np.ndarray:
    """Geometric progression from N_min to N_max."""
    if self.n_levels == 1:
        return np.array([self.base_resolution], dtype=np.int32)
    b = (self.finest_resolution / self.base_resolution) ** (1.0 / (self.n_levels - 1))
    return np.floor(
        self.base_resolution * (b ** np.arange(self.n_levels))
    ).astype(np.int32)
```

- Обчислюємо резолюції всіх L рівнів геометричною прогресією від $N_{\min}$ до $N_{\max}$.
- **Особливий випадок:** якщо тільки 1 рівень, повертаємо одну резолюцію.
- **`b`** — основа геометричної прогресії: $b = (N_{\max} / N_{\min})^{1/(L-1)}$. Для $N_{\min}=8$, $N_{\max}=128$, $L=8$: $b \approx 1.486$.
- **`np.arange(8)`** — масив `[0, 1, 2, 3, 4, 5, 6, 7]`.
- **`b ** np.arange(8)`** — `[b0, b1, b2, ..., b7]` поелементно.

- `base * (b ** arange)` — `[N_min*b^0, N_min*b^1, ...] = [8, 11.89, 17.67, ..., 128]`.
- `np.floor(...).astype(np.int32)` — округлення вниз і конвертація в цілі: `[8, 11, 17, 26, 39, 58, 86, 128]`.

Клас моделі — конструктор (рядки 42–72)

```
class MultiResHashSDF(nn.Module):
    """point  $\in [-1,1]^3 \rightarrow \text{scalar SDF}$ ."""

    def __init__(self, cfg: HashConfig = HashConfig()):
        super().__init__()
        self.cfg = cfg
        T, F, L = cfg.T, cfg.n_features, cfg.n_levels
```

- **`nn.Module`** — базовий клас усіх PyTorch-моделей. Він автоматично відстежує параметри, дозволяє переміщати на пристрої (CPU/GPU/MPS), серіалізувати тощо.
- **`super().__init__()`** — обов'язковий виклик конструктора батьківського класу. Без нього PyTorch не знатиме, що це Module.
- **`self.cfg = cfg`** — зберігаємо конфігурацію, щоб потім мати доступ.
- **`T, F, L = ...`** — розпакування для скорочення подальших рядків.

```
# Hash table per level. Small init keeps training stable.
self.features = nn.Parameter(torch.empty(L, T, F).uniform_(-1e-4, 1e-4))
```

- **`torch.empty(L, T, F)`** — створює неініціалізований тензор форми $(L, T, F) = (8, 8192, 2)$. Швидко (без обнулення).
- **`.uniform_(-1e-4, 1e-4)`** — заповнює його рівномірно з $[-10^{-4}, 10^{-4}]$. Підкреслення в кінці методу означає "in-place" — модифікує сам тензор.
- **`nn.Parameter(...)`** — спеціальний підклас тензора, який PyTorch автоматично включає до `model.parameters()`. Це найважливіша магія — без неї оптимізатор не знатиме, що ці ваги потрібно тренувати.
- **Чому такі дрібні початкові значення?** Бо при більших початкових значеннях модель з самого початку дає сильні (але випадкові) передбачення, і градієнтному спуску важче їх "відштовхнути". Маленькі значення близькі до "ще нічого не вивчив".

```

in_dim = L * F
if cfg.hidden > 0:
    self.decoder = nn.Sequential(
        nn.Linear(in_dim, cfg.hidden),
        nn.ReLU(),
        nn.Linear(cfg.hidden, 1),
    )
else:
    self.decoder = nn.Linear(in_dim, 1)
    nn.init.zeros_(self.decoder.bias)
    nn.init.normal_(self.decoder.weight, std=1e-2)

```

- `in_dim = L * F = 16` — це довжина вектора, який ми отримуємо після конкатенації feature-векторів з усіх рівнів.
- **Випадок `cfg.hidden > 0`**: створюємо мінімальний MLP. `nn.Sequential` — це список модулів, які застосовуються один за одним.
- `nn.Linear(16, 16)` — повнозв'язний шар.
- `nn.ReLU()` — нелінійність.
- `nn.Linear(16, 1)` — фінальний шар у скалярний вихід.
- **Випадок `cfg.hidden == 0`**: лінійний декодер (без нелінійності). Окремо ініціалізуємо `bias = 0` і `weight ~ N(0, 0.012)` — це обережна ініціалізація, типова для лінійних шарів.

```

self.register_buffer("resolutions",
                    torch.from_numpy(cfg.resolutions).to(torch.int64))

```

- `register_buffer` — спеціальний механізм PyTorch для **констант моделі**. На відміну від `nn.Parameter`, `buffer` **не** тренується (немає градієнтів), але **переноситься** з моделлю на інший пристрій і **зберігається** при серіалізації.
- Тут ми зберігаємо резольюції як `buffer`, бо вони потрібні в `forward` пасі, і має сенс мати їх на тому ж пристрої, де модель.

Властивість для підрахунку параметрів (рядки 74–76)

```

@property
def n_params(self) -> int:
    return sum(p.numel() for p in self.parameters())

```

- `self.parameters()` — generator усіх параметрів моделі (тих, що в `nn.Parameter`).
- `p.numel()` — кількість елементів у параметрі (наприклад, $L \cdot T \cdot F = 131072$ для features).
- **Сума** — загальна кількість параметрів моделі.

Forward pass (рядки 78–121)

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    cfg = self.cfg
    B = x.shape[0]
```

- **`x.shape[0]`** — batch-розмір (кількість точок). **`x`** має форму $(B, 3)$.

```
u = (x.clamp(-cfg.bound, cfg.bound) + cfg.bound) / (2.0 * cfg.bound)
```

- **`x.clamp(-1, 1)`** — обрізає кожну координату до $[-1, 1]$.
- **`(x + 1) / 2`** — переводить з $[-1, 1]$ у $[0, 1]$. Це поелементна операція над тензором.
- Тепер **`u`** має форму $(B, 3)$ зі значеннями в $[0, 1]$.

```
feats = []
for L_i in range(cfg.n_levels):
    N = self.resolutions[L_i].item()
    scaled = u * (N - 1)
    base = torch.floor(scaled).to(torch.int64)
    frac = scaled - base.to(scaled.dtype)
```

- Цикл по рівнях (типово 8).
- **`self.resolutions[L_i].item()`** — Python-int з тензора. **`.item()`** потрібно, бо ми використовуємо це число у звичайному множенні з тензором.
- **`u * (N - 1)`** — масштабуємо координати до сітки. Якщо $u_x = 0.6$ і $N = 58$, то $\text{scaled}_x = 0.6 \cdot 57 = 34.2$.
- **`torch.floor(scaled).to(torch.int64)`** — округлюємо вниз. Маємо тензор з цілими координатами кутів вокселя.
- **`scaled - base.to(scaled.dtype)`** — дробова частина в $[0, 1]$. Це наші **`tx`**, **`ty`**, **`tz`** для трилінійної інтерполяції.

```
x0, y0, z0 = base[:, 0], base[:, 1], base[:, 2]
x1, y1, z1 = x0 + 1, y0 + 1, z0 + 1
```

- **`base[:, 0]`** — береться x-координата всіх точок батчу (форма $(B,)$).
- **`(x0, y0, z0)`** — індекси "нижнього-лівого-заднього" кута комірки.
- **`(x1, y1, z1)`** — індекси "верхнього-правого-переднього" кута. Це 7 одиниць більше, бо комірка має розмір 1 у грид-координатах.

```

T = cfg.T
h000 = _spatial_hash(x0, y0, z0, T)
h100 = _spatial_hash(x1, y0, z0, T)
h010 = _spatial_hash(x0, y1, z0, T)
h110 = _spatial_hash(x1, y1, z0, T)
h001 = _spatial_hash(x0, y0, z1, T)
h101 = _spatial_hash(x1, y0, z1, T)
h011 = _spatial_hash(x0, y1, z1, T)
h111 = _spatial_hash(x1, y1, z1, T)

```

- **8 хешів** для 8 кутів вокселя. Нумерація $hXYZ$ — X для $x0$ чи $x1$, аналогічно для Y, Z . Тобто $h110 = (x1, y1, z0)$.
- Кожен $hXYZ$ — це тензор форми $(B,)$ з індексами в хеш-таблицю.

```

tbl = self.features[L_i]
f000, f100 = tbl[h000], tbl[h100]
f010, f110 = tbl[h010], tbl[h110]
f001, f101 = tbl[h001], tbl[h101]
f011, f111 = tbl[h011], tbl[h111]

```

- **self.features[L_i]** — береться хеш-таблиця для поточного рівня. Форма $(T, F) = (8192, 2)$.
- **tbl[h000]** — це **fancy indexing** PyTorch: для кожного індексу в $h000$ беремо відповідний рядок з таблиці. Результат — тензор форми (B, F) .

```
tx = frac[:, 0:1]; ty = frac[:, 1:2]; tz = frac[:, 2:3]
```

- **frac[:, 0:1]** — це tx як тензор форми $(B, 1)$, а не $(B,)$.
- **Чому 0:1, а не просто 0?** Бо для broadcasting в подальших операціях треба, щоб tx мав другу вимір (1), яка може бути розгорнута до F при множенні з тензором форми (B, F) .

```

c00 = f000 * (1 - tx) + f100 * tx
c10 = f010 * (1 - tx) + f110 * tx
c01 = f001 * (1 - tx) + f101 * tx
c11 = f011 * (1 - tx) + f111 * tx
c0  = c00 * (1 - ty) + c10 * ty
c1  = c01 * (1 - ty) + c11 * ty
f   = c0  * (1 - tz) + c1  * tz
feats.append(f)

```

- **Трилінійна інтерполяція в три проходи** (по X , потім Y , потім Z).
- На виході f — тензор форми (B, F) для поточного рівня.

- `feats.append(f)` — додаємо до списку результатів.

```
h = torch.cat(feats, dim=-1)
return self.decoder(h).squeeze(-1)
```

- `torch.cat(feats, dim=-1)` — конкатенація по останньому виміру. Якщо в `feats` 8 тензорів форми (B, F) , результат — $(B, L \cdot F) = (B, 16)$.
- `self.decoder(h)` — пропускаємо через MLP, отримуємо тензор $(B, 1)$.
- `.squeeze(-1)` — прибирає останню вимір розміру 1, отримуємо $(B,)$.

Властивість розміру моделі (рядки 123–127)

```
@property
def model_size_bytes(self) -> int:
    feat_b = self.features.numel() * 2
    dec_b = sum(p.numel() * 4 for p in self.decoder.parameters())
    return feat_b + dec_b
```

- Обчислюємо передбачуваний розмір при експорті.
- `* 2` — бо features зберігаються в `fp16` (2 байти на число).
- `* 4` — бо декодер зберігається в `fp32` (4 байти).
- **Сума** — загальний розмір.

Експорт у .npz (рядки 130–159)

```
def export_npz(self, path: str | Path) -> None:
    import json
    feats = self.features.detach().cpu().to(torch.float16).numpy()
    res = self.resolutions.detach().cpu().to(torch.int32).numpy()
    cfg_json = json.dumps(self.cfg.__dict__)
    out = dict(features=feats, resolutions=res, cfg=np.array(cfg_json))
```

- `import json` — імпортуємо локально, щоб не засмічувати модуль на верхньому рівні.
- `.detach()` — від'єднує від обчислювального графа (бо ми зберігаємо, а не тренуємо).
- `.cpu()` — на випадок, якщо ми були на MPS/GPU, переносимо на CPU.
- `.to(torch.float16)` — конвертуємо до 16-бітних чисел. Це **критична економія пам'яті** — половина розміру без помітних втрат якості.
- `.numpy()` — переводимо в numpy-масив.
- `json.dumps(self.cfg.__dict__)` — серіалізуємо конфігурацію як JSON-рядок. `__dict__` повертає словник з полями dataclass.

- `np.array(cfg_json)` — обгортаємо в numpy-масив, щоб можна було зберегти разом з іншими.

```
if isinstance(self.decoder, nn.Linear):
    out['decoder_W'] = self.decoder.weight.detach().cpu().to(torch.float32).numpy()
    out['decoder_b'] = self.decoder.bias.detach().cpu().to(torch.float32).numpy()
else:
    l1, _, l2 = self.decoder # Linear, ReLU, Linear
    out['W1'] = l1.weight.detach().cpu().to(torch.float32).numpy()
    out['b1'] = l1.bias.detach().cpu().to(torch.float32).numpy()
    out['W2'] = l2.weight.detach().cpu().to(torch.float32).numpy()
    out['b2'] = l2.bias.detach().cpu().to(torch.float32).numpy()
np.savez_compressed(path, **out)
```

- **Різні гілки** залежно від типу декодера:
- Лінійний (`hidden=0`) — зберігаємо одну матрицю `W` і вектор `b`.
- MLP (`hidden=16`) — розпаковуємо `Sequential` на 3 модулі (Linear, ReLU, Linear), зберігаємо `W1`, `b1`, `W2`, `b2`.
- `l1, _, l2 = self.decoder` — Python розпаковує `Sequential` як ітерабельний об'єкт. ReLU йде в `_` — її не потрібно зберігати, бо вона не має параметрів.
- `np.savez_compressed(path, **out)` — зберігає словник у стиснутий `.npz` файл. `**out` розпаковує словник як кейворд-аргументи.

Завантаження з `.npz` (рядки 161–177)

```
@staticmethod
def from_npz(path: str | Path) -> "MultiResHashSDF":
    import json
    z = np.load(path, allow_pickle=False)
    cfg = HashConfig(**json.loads(str(z["cfg"])))
    model = MultiResHashSDF(cfg)
```

- `@staticmethod` — метод викликається без `self`, як `MultiResHashSDF.from_npz('path')`.
- `allow_pickle=False` — безпекова опція; ми не зберігаємо складних Python-об'єктів.
- `json.loads(str(z["cfg"]))` — парсимо JSON-рядок назад у словник.
- `HashConfig(**dict)` — конструємо `HashConfig` з кейворд-аргументів.

```

with torch.no_grad():
    model.features.copy_(torch.from_numpy(z["features"].astype(np.float32)))
    if cfg.hidden == 0:
        model.decoder.weight.copy_(torch.from_numpy(z["decoder_W"]))
        model.decoder.bias.copy_(torch.from_numpy(z["decoder_b"]))
    else:
        l1, _, l2 = model.decoder
        l1.weight.copy_(torch.from_numpy(z["W1"]))
        l1.bias.copy_(torch.from_numpy(z["b1"]))
        l2.weight.copy_(torch.from_numpy(z["W2"]))
        l2.bias.copy_(torch.from_numpy(z["b2"]))
return model

```

- **with torch.no_grad():** — контекст без обчислення градієнтів. Економить пам'ять і час при копіюванні ваг.
- **.astype(np.float32)** — конвертуємо fp16 → fp32 для математики (CPU/GPU нативно рахують fp32).
- **.copy_(...)** — копіює дані в існуючий тензор (in-place, відмічається підкресленням).
- Знову різні гілки для лінійного / MLP декодера.

Файл 2: sdf_inference.py

Цей файл — **окремий швидкий inference-кернел на Numba**, незалежний від PyTorch. Завантажує `.npz`, експортовані `sdf_model.py`, і обчислює SDF на CPU в нативній швидкості.

Імпорти (рядки 1–7)

```

import json
from pathlib import Path
from typing import Optional

import numpy as np
import numba
from numba import njit, prange

```

- **Optional** — анотація для аргументів, які можуть бути `None`. Тут не використовується, можна було б видалити.
- **numba** — JIT-компілятор. Перетворює Python-функції на оптимальний машинний код.

- **njit** — декоратор для функцій, що мають бути скомпільовані "no Python" (без Python-runtime).
- **prange** — паралельна версія **range** для багатопотокових циклів у numba.

Константи хешу (рядки 9–11)

```
_P0 = np.int64(1)
_P1 = np.int64(2_654_435_761)
_P2 = np.int64(805_459_861)
```

- Ті ж самі прості числа, що в `sdf_model.py`.
- **np.int64(...)** — явна типізація. Чому? Бо numba дуже строго типізує: якщо ми передамо ці константи в njit-функцію, numba має знати їх тип. **np.int64** — це 64-бітне ціле зі знаком, достатньо великого для всіх множень.

Функція хешу (рядки 14–17)

```
@njit(cache=True, fastmath=True, inline='always')
def _hash3(ix: int, iy: int, iz: int, T: int) -> int:
    h = (ix * _P0) ^ (iy * _P1) ^ (iz * _P2)
    return h & (T - 1)
```

- **@njit** — JIT-декоратор. Numba скомпілює цю функцію при першому виклику.
- **cache=True** — скомпільований код зберігається на диск, щоб наступний запуск не перекомпілював.
- **fastmath=True** — дозволяє компілятору переставляти арифметичні операції для швидкості. Може дати мікроскопічні розбіжності в молодших цифрах — для нас неважливо.
- **inline='always'** — підказка компілятору **завжди підставляти тіло цієї функції** в місце виклику, а не робити справжній виклик. Економить overhead функції-виклику.
- **h & (T - 1)** — bitmask замість modulo. Працює тільки якщо **T** — степінь двійки. У нас **T = 2¹³** або **2¹⁴**, обидва підходять. На M4 це **~5x швидше** за modulo.

Forward для однієї точки (рядки 20–97)

Це **серце** inference-кернелу. Він точно відтворює forward з `sdf_model.py`, але в numba.

```
@jit(cache=True, fastmath=True)
def _forward_one(x: float, y: float, z: float,
                features: np.ndarray,      # (L, T, F) float32
                res: np.ndarray,           # (L,) int64
                W1: np.ndarray, b1: np.ndarray, # decoder hidden
                W2: np.ndarray, b2: np.ndarray, # decoder output
                has_hidden: bool) -> float:
```

- Numba не підтримує **class methods** (ефективно), тому ми передаємо всі ваги моделі як окремі аргументи.
- **has_hidden: bool** — runtime-флаг, чи модель з MLP-декодером чи лінійним. Numba спеціалізує код для кожного значення цього флагу.

```
L = features.shape[0]
T = features.shape[1]
F = features.shape[2]
in_dim = L * F
```

- Розпаковуємо розміри з форми features.
- Тут **L**, **T**, **F** — локальні змінні, не глобальні константи. Numba знає типи з контексту.

```
if x < -1.0: x = -1.0
elif x > 1.0: x = 1.0
if y < -1.0: y = -1.0
elif y > 1.0: y = 1.0
if z < -1.0: z = -1.0
elif z > 1.0: z = 1.0
ux = (x + 1.0) * 0.5
uy = (y + 1.0) * 0.5
uz = (z + 1.0) * 0.5
```

- **Clamp + map до $[0, 1]$** . У PyTorch це робилося через `.clamp` + ділення; тут — звичайними if-else, бо numba не має векторного `clamp` для скалярів.
- *** 0.5** замість **/ 2.0** — деякі компілятори генерують ідентичний код, але множення концептуально швидше для деяких архітектур.

```
h_buf = np.empty(in_dim, dtype=np.float32)
```

- **Локальний буфер** для конкатенованих фіч. У numba `np.empty` робить алокацію на стеку (якщо розмір відомий), що дуже швидко.
- **in_dim = L * F = 16** — маленький буфер.

```

for Li in range(L):
    N = res[Li]
    sx = ux * (N - 1.0)
    sy = uy * (N - 1.0)
    sz = uz * (N - 1.0)
    ix = np.int64(sx); iy = np.int64(sy); iz = np.int64(sz)
    fx = sx - ix; fy = sy - iy; fz = sz - iz

```

- Цикл по рівнях.
- **np.int64(sx)** — конверсія в ціле з округленням до нуля (як `floor` для додатних чисел).
- Це робиться по три координати окремо, бо numba любить скаляри.

```

h000 = _hash3(ix,    iy,    iz,    T)
h100 = _hash3(ix + 1, iy,    iz,    T)
h010 = _hash3(ix,    iy + 1, iz,    T)
h110 = _hash3(ix + 1, iy + 1, iz,    T)
h001 = _hash3(ix,    iy,    iz + 1, T)
h101 = _hash3(ix + 1, iy,    iz + 1, T)
h011 = _hash3(ix,    iy + 1, iz + 1, T)
h111 = _hash3(ix + 1, iy + 1, iz + 1, T)

```

- 8 хешів — як у PyTorch, але викликається інлайн-функція `_hash3`.

```

omfx = 1.0 - fx; omfy = 1.0 - fy; omfz = 1.0 - fz
for Fi in range(F):
    f000 = features[Li, h000, Fi]
    f100 = features[Li, h100, Fi]
    f010 = features[Li, h010, Fi]
    f110 = features[Li, h110, Fi]
    f001 = features[Li, h001, Fi]
    f101 = features[Li, h101, Fi]
    f011 = features[Li, h011, Fi]
    f111 = features[Li, h111, Fi]

    c00 = f000 * omfx + f100 * fx
    c10 = f010 * omfx + f110 * fx
    c01 = f001 * omfx + f101 * fx
    c11 = f011 * omfx + f111 * fx
    c0  = c00 * omfy + c10 * fy
    c1  = c01 * omfy + c11 * fy
    h_buf[Li * F + Fi] = c0 * omfz + c1 * fz

```

- **omfx = 1.0 - fx** — попередньо обчислюємо "один мінус fx", щоб не рахувати тричі.
- Внутрішній цикл `for Fi in range(F)` — по 2 виміри feature-вектора.

- Все та сама трилінійна інтерполяція, але писана скалярно. Numba буде векторизувати це автоматично.

```

if has_hidden:
    hidden = W1.shape[0]
    out = b2[0]
    for j in range(hidden):
        acc = b1[j]
        for i in range(in_dim):
            acc += W1[j, i] * h_buf[i]
        if acc < 0.0:
            acc = 0.0
        out += W2[0, j] * acc
    return out
else:
    out = b2[0]
    for i in range(in_dim):
        out += W2[0, i] * h_buf[i]
    return out

```

- **MLP-декодер вручну**: цикл по hidden нейронах, всередині кожного — добуток $W1[j,:]$ на h_buf , потім ReLU (`if acc < 0: acc = 0`), потім додаємо до виходу зі вагою $W2[0, j]$.
- **Лінійний декодер** — просто одне множення матриця-вектор.

Batch forward (рядки 100–111)

```

@jit(cache=True, fastmath=True, parallel=True)
def _forward_batch(pts: np.ndarray,          # (B, 3) float32
                   features: np.ndarray,
                   res: np.ndarray,
                   W1: np.ndarray, b1: np.ndarray,
                   W2: np.ndarray, b2: np.ndarray,
                   has_hidden: bool,
                   out: np.ndarray) -> None:
    B = pts.shape[0]
    for i in prange(B):
        out[i] = _forward_one(pts[i, 0], pts[i, 1], pts[i, 2],
                               features, res, W1, b1, W2, b2, has_hidden)

```

- **parallel=True** — дозволяє numba розпаралелити цикл.
- **prange(B)** — паралельний range. Numba автоматично розбиває $[0, B)$ на чанки і виконує їх на кількох CPU-потоках.
- Результат пишеться в **out**-параметр, наданий зовні. Це уникає алокації всередині — кращий для продуктивності.

Бенчмарк-функція (рядки 114–124)

```
@jit(cache=True, fastmath=True)
def _forward_loop_bench(x: float, y: float, z: float,
                        n_iters: int,
                        features: np.ndarray, res: np.ndarray,
                        W1: np.ndarray, b1: np.ndarray,
                        W2: np.ndarray, b2: np.ndarray,
                        has_hidden: bool) -> float:
    acc = 0.0
    for _ in range(n_iters):
        acc += _forward_one(x, y, z, features, res, W1, b1, W2, b2, has_hidden)
    return acc
```

- Виконує `_forward_one` багато разів у тісному циклі.
- `acc += ...` — накопичуємо суму, щоб компілятор **не викинув** цикл як "мертвий код" (бо результат залежить від суми).
- Виклик з Python: міряємо час, ділимо на `n_iters` → отримуємо наносекунди на запит **без Python-overhead**.

Клас SDFInference (рядки 127–175)

```
class SDFInference:

    def __init__(self, npz_path: str | Path):
        z = np.load(npz_path, allow_pickle=False)
        cfg = json.loads(str(z['cfg']))
        self.cfg = cfg
        self.has_hidden = cfg.get('hidden', 0) > 0

        self.features = z['features'].astype(np.float32)
        self.res = z['resolutions'].astype(np.int64)
```

- Завантаження з `.npz` — той самий формат, що експортує `sdf_model.py`.
- `.astype(np.float32)` — конвертуємо fp16 → fp32 для нативної швидкості CPU.

```

if self.has_hidden:
    self.W1 = z['W1'].astype(np.float32)
    self.b1 = z['b1'].astype(np.float32)
    self.W2 = z['W2'].astype(np.float32)
    self.b2 = z['b2'].astype(np.float32)
else:
    self.W1 = np.zeros((1, 1), dtype=np.float32)
    self.b1 = np.zeros((1, 1), dtype=np.float32)
    self.W2 = z['decoder_W'].astype(np.float32)
    self.b2 = z['decoder_b'].astype(np.float32)

```

- Завантажуємо ваги декодера.
- **else** -гілка: для лінійного декодера ставимо dummy W1/b1 (numba потрібно щось передати), а реальні ваги йдуть у W2/b2.

```

def query_one(self, x: float, y: float, z: float) -> float:
    return float(_forward_one(x, y, z, self.features, self.res,
                              self.W1, self.b1, self.W2, self.b2,
                              self.has_hidden))

```

- Просто проксі до `_forward_one`. `float(...)` потрібно, бо numba може повернути numpy-скаляр.

```

def __call__(self, pts: np.ndarray) -> np.ndarray:
    pts = np.ascontiguousarray(pts, dtype=np.float32)
    out = np.empty(pts.shape[0], dtype=np.float32)
    _forward_batch(pts, self.features, self.res,
                   self.W1, self.b1, self.W2, self.b2,
                   self.has_hidden, out)
    return out

```

- **np.ascontiguousarray(...)** — гарантує, що масив у безперервній C-style розкладці пам'яті. Numba з не-contiguous масивами працює погано.
- **np.empty** — алокує вихідний масив (без ініціалізації).
- Виклик `_forward_batch`, який запише результат в `out`.

```

def warmup(self) -> None:
    _ = self.query_one(0.0, 0.0, 0.0)
    _ = self(np.zeros((4, 3), dtype=np.float32))

```

- **Перший виклик numba-функції — повільний** (компіляція ~1 сек). `warmup` робить ці виклики заздалегідь, щоб справжні запити були швидкими.


```
def bench_single_point(self, n_iters: int = 100_000) -> float:
    import time
    _forward_loop_bench(0.1, 0.2, 0.3, 100, self.features, self.res,
                        self.W1, self.b1, self.W2, self.b2, self.has_hidden)
    t0 = time.perf_counter()
    _forward_loop_bench(0.1, 0.2, 0.3, n_iters, self.features, self.res,
                        self.W1, self.b1, self.W2, self.b2, self.has_hidden)
    dt = time.perf_counter() - t0
    return dt / n_iters * 1e9
```

- Спочатку warmup (100 ітерацій, щоб triggerнути компіляцію).
- Потім справжній бенчмарк (typically 100K ітерацій).
- `time.perf_counter()` — найточніший таймер Python.
- Ділимо на `n_iters` і множимо на 10^9 → отримуємо наносекунди.

Файл 3: train.ipynb

Це Jupyter-блокнот, що містить EDA, тренування, оцінку та бенчмарки. Розглянемо по клітинках.

Клітинка 0: Імпорти і конфігурація

```
import os, gc, json, time, resource
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

import trimesh
import pysdf
import torch

from sdf_model import MultiResHashSDF, HashConfig
```

- **gc** — garbage collector. Викликаємо `gc.collect()` після того, як прибираємо великі об'єкти, щоб звільнити пам'ять.
- **resource** — Unix-модуль для отримання resident memory size (RSS).
- **trimesh** — бібліотека для роботи з 3D-mesh.
- **pysdf** — швидкий C++ обчислювач SDF.

- `from sdf_model import ...` — наш кастомний модуль.

```
MESH_DIR = Path('test_task_meshes')
MODELS_DIR = Path('models'); MODELS_DIR.mkdir(exist_ok=True)
```

- Шляхи до директорій.
- `.mkdir(exist_ok=True)` — створює директорію, але не помиляється, якщо вона вже існує.

```
RUN_TRAIN_ALL = True
RUN_EVAL_ALL = True
RUN_RETRAIN_HARD = False
RUN_BENCH = True
```

- **Прапорці-перемикачі.** Безпековий механізм: жодна важка операція не виконується автоматично. Користувач має явно встановити `True`.

```
DEVICE = torch.device('cpu')
```

- На M4 MPS (Metal Performance Shaders) має великий dispatch-overhead для маленьких моделей. CPU фактично швидший.

```
def mem_mb() -> float:
    rss = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss
    return rss / (1024**2) if os.uname().sysname == 'Darwin' else rss / 1024
```

- Отримує **резидентну пам'ять** (RSS) поточного процесу.
- `ru_maxrss` — це максимум, на macOS у байтах, на Linux — у кілобайтах. Тому конвертація відрізняється.

```
mesh_files = sorted(MESH_DIR.glob('*.obj'), key=lambda p: int(p.stem))
print(f'Found {len(mesh_files)} meshes | device: {DEVICE} | RSS: {mem_mb():.0f} MB')
```

- `.glob('*.obj')` — generator усіх `.obj` файлів.
- `key=lambda p: int(p.stem)` — сортуємо по **числовому** імені, а не лексикографічно. Тобто `0.obj`, `1.obj`, `2.obj`, `10.obj` (а не `0`, `1`, `10`, `2`).
- `p.stem` — ім'я файлу без розширення.

Клітинка 2: Утиліти для роботи з mesh

```
def load_mesh(path: Path) -> trimesh.Trimesh:
    m = trimesh.load(path, force='mesh')
    if isinstance(m, trimesh.Scene):
        m = trimesh.util.concatenate([
            trimesh.Trimesh(vertices=g.vertices, faces=g.faces) for g in m.geometry.values()])
    return m
```

- **force='mesh'** — каже trimesh повернути `Trimesh`, а не `Scene` (де можуть бути кілька об'єктів).
- Але іноді все одно повертається `Scene`, тому є fallback: **конкатенуємо** всі під-mesh у один.

```
def normalize_mesh(m: trimesh.Trimesh, padding: float = 0.95) -> trimesh.Trimesh:
    """Center at origin, isotropically scale so the bbox sits in [-padding, padding]."""
    m = m.copy()
    center = (m.bounds[0] + m.bounds[1]) / 2.0
    m.apply_translation(-center)
    m.apply_scale(padding / np.abs(m.bounds).max())
    return m
```

- **m.copy()** — копія, щоб не модифікувати оригінал.
- **m.bounds** — bounding box як `[[xmin, ymin, zmin], [xmax, ymax, zmax]]`.
- **center = (min + max) / 2** — геометричний центр bbox.
- **apply_translation(-center)** — зсуваємо так, що центр в $(0, 0, 0)$.
- **np.abs(m.bounds).max()** — найдалі координата по модулю. У нормалізованому mesh (центрованому), bounds симетричні, тому це теж напіврозмах bbox.
- **padding / ...** — коефіцієнт масштабування. Якщо `padding = 0.95`, mesh буде у $[-0.95, 0.95]^3$, з запасом до меж.

Клітинка 4: EDA — статистика по mesh-ам

```
rows = []
for p in mesh_files:
    m = load_mesh(p)
    bbox = m.bounds; extents = bbox[1] - bbox[0]
    rows.append({
        'id': int(p.stem),
        'n_vertices': len(m.vertices),
        'n_faces': len(m.faces),
        'watertight': bool(m.is_watertight),
        'area': float(m.area),
        'volume': float(m.volume) if m.is_watertight else float('nan'),
        'max_extent': float(extents.max()),
        'volume_frac': float(m.volume) / (extents[0]*extents[1]*extents[2]) if m.is_watertight else float('nan')
    })
    del m; gc.collect()
```

- **extents = bbox[1] - bbox[0]** — розміри bbox по кожній осі.
- **m.is_watertight** — чи mesh є "водотривкий" (без дірок). Якщо ні, **volume не визначено**.
- **del m; gc.collect()** — явно прибираємо великий об'єкт. Без цього 50 mesh-ів накопичуються в пам'яті.

```
df = pd.DataFrame(rows).sort_values('id').reset_index(drop=True)
print(f'Watertight: {df["watertight"].sum()}/{len(df)} | RSS: {mem_mb():.0f} MB')
df.describe()
```

- Конструюємо pandas DataFrame з ряду словників.
- **.describe()** — стандартна статистика (mean, std, min, max, квартали).

Клітинка 5: Гістограми

```
fig, axes = plt.subplots(1, 3, figsize=(13, 3))
axes[0].hist(df['n_faces'], bins=25); axes[0].set_title('#faces')
axes[1].hist(df['volume_frac'].dropna(), bins=25); axes[1].set_title('volume / bbox vol')
axes[2].hist(df['area'], bins=25); axes[2].set_title('surface area')
plt.tight_layout(); plt.show()
```

- **plt.subplots(1, 3, figsize=(13, 3))** — 3 субплоти в одному рядку.
- **axes[i].hist(...)** — гістограма для кожного.
- **.dropna()** — пропускаємо NaN-значення (для non-watertight mesh-ів).

Клітинка 7: Візуалізація

```
fig = plt.figure(figsize=(12, 8))
for i, mid in enumerate([0, 5, 10, 18, 26, 49]):
    m = normalize_mesh(load_mesh(MESH_DIR / f'{mid}.obj'))
    pts, _ = trimesh.sample.sample_surface(m, 1500)
    ax = fig.add_subplot(2, 3, i+1, projection='3d')
    ax.scatter(pts[:,0], pts[:,1], pts[:,2], s=0.5)
    ax.set_title(f'mesh {mid} ({len(m.faces)} faces)')
    ax.set_xlim(-1,1); ax.set_ylim(-1,1); ax.set_zlim(-1,1)
    ax.set_box_aspect((1,1,1))
    del m, pts; gc.collect()
plt.tight_layout(); plt.show()
```

- Беремо 6 mesh-ів, для кожного семплимо 1500 точок з поверхні, скетеримо їх в 3D.
- **add_subplot(2, 3, i+1, projection='3d')** — 2x3 сітка, поточна позиція i+1 (1-indexed), 3D-вісі.
- **set_box_aspect((1,1,1))** — рівні масштаби по всіх осях. Без цього 3D-куб виглядав би розтягнутим.

Клітинка 9: Функції тренування

```
def compute_f1(pred: np.ndarray, gt_inside: np.ndarray) -> float:
    pred_inside = pred < 0
    tp = np.logical_and(pred_inside, gt_inside).sum()
    fp = np.logical_and(pred_inside, ~gt_inside).sum()
    fn = np.logical_and(~pred_inside, gt_inside).sum()
    if tp + fp + fn == 0:
        return 1.0
    prec = tp / max(tp + fp, 1); rec = tp / max(tp + fn, 1)
    return 0.0 if prec + rec == 0 else float(2 * prec * rec / (prec + rec))
```

- **F1** — гармонійне середнє precision і recall.
- **TP** (true positive): передбачили "всередині", і насправді всередині.
- **FP** (false positive): передбачили "всередині", але насправді зовні.
- **FN** (false negative): передбачили "зовні", але насправді всередині.
- **max(..., 1)** — захист від ділення на нуль.
- **F1 = 2 · prec · rec / (prec + rec)**.

```
def train_one_mesh(mid: int,
                  n_iters: int = 2500,
                  cfg: HashConfig = None,
                  lr: float = 5e-3,
                  per_step=dict(surface=512, near_1e2=4000, near_1e3=2000, uniform=1024),
                  eval_every: int = 250,
                  early_stop_f1: float = 0.96,
                  verbose: bool = False):
```

- **Параметри з дефолтами** — дозволяють викликати з мінімумом аргументів.
- **cfg=None** означає "використовуй HashConfig() з дефолтами".
- **per_step** — словник з квотами на batch. Сумарно ~7500 точок на крок.

```
mesh = normalize_mesh(load_mesh(MESH_DIR / f'{mid}.obj'))
sdf_fn = pysdf.SDF(mesh.vertices, mesh.faces)
```

- Завантаження і нормалізація mesh.
- **pysdf.SDF(vertices, faces)** — створює об'єкт, який вміє швидко обчислювати SDF. Внутрішньо будує AABB-tree.

```
rng_e = np.random.default_rng(999 + mid); n_e = 10_000
base_e = trimesh.sample.sample_surface(mesh, n_e)[0]
eval_pts = np.concatenate([
    (base_e + rng_e.normal(0, 1e-2, (n_e, 3))).astype(np.float32),
    rng_e.uniform(-1, 1, (n_e, 3)).astype(np.float32)], axis=0)
eval_sdf = (-sdf_fn(eval_pts)).astype(np.float32)
```

- **np.random.default_rng(seed)** — сучасний шлях створення PRNG в numpy. Кожен mesh має унікальне сид (999 + mid) для відтворюваності.
- **trimesh.sample.sample_surface(mesh, n)** — повертає $(n, 3)$ масив точок, рівномірно розподілених по площі поверхні.
- **[0]** — sample_surface повертає tuple (points, face_ids), нам потрібні тільки точки.
- **Eval-набір** — 10k точок поверхня+шум + 10k рівномірно з куба. **Незалежний** від тренувальних точок (інший сид).
- **-sdf_fn(...)** — pysdf повертає positive=inside, ми перевертаємо знак до стандартної конвенції (negative=inside).

```
eval_gi_near = eval_sdf[:n_e] < 0
eval_gi_unif = eval_sdf[n_e:] < 0
eval_pts_t = torch.from_numpy(eval_pts).to(DEVICE)
```

- **gi** — "ground inside" — boolean масив, який є True, якщо точка всередині.

- **.to(DEVICE)** — на CPU (у нас).

```
torch.manual_seed(mid)
model = MultiResHashSDF(cfg if cfg is not None else HashConfig()).to(DEVICE)
opt = torch.optim.Adam(model.parameters(), lr=lr)
```

- **torch.manual_seed(mid)** — для відтворюваності ініціалізації ваг.
- Створюємо модель з конфігурації.
- **Adam** — оптимізатор. **model.parameters()** — генератор усіх тренувальних параметрів (features + W1, b1, W2, b2).

```
rng = np.random.default_rng(42 + mid)
Q = per_step; history = []
for it in range(n_iters):
```

- Інший RNG для тренувальних точок (відрізняється від eval-RNG).

```
base = trimesh.sample.sample_surface(mesh, Q['surface'] + Q['near_1e2'] + Q['near_1e3'])
p_s = base[Q['surface']]
p_n2 = base[Q['surface']:Q['surface']+Q['near_1e2']] + rng.normal(0, 1e-2, (Q['near_1e2']
p_n3 = base[-Q['near_1e3']:] + rng.normal(0, 1e-3, (Q['near_1e3'], 3))
p_u = rng.uniform(-1, 1, (Q['uniform'], 3))
pts_np = np.concatenate([p_s, p_n2, p_n3, p_u], axis=0).astype(np.float32)
```

- **One-shot sampling:** одним викликом отримуємо всі точки з поверхні, потім розподіляємо.
- **p_s** — surface точки (без шуму).
- **p_n2** — surface + шум $\mathcal{N}(0, 0.01^2)$ — для тренування біля поверхні з тим самим масштабом, що eval.
- **p_n3** — surface + шум $\mathcal{N}(0, 0.001^2)$ — дрібніший шум, для дрібніших деталей.
- **p_u** — рівномірно в кубі.
- **base[-Q['near_1e3']:]** — останні N точок з base. Це може **перекриватися** з ранніми сегментами, але це не проблема — все одно ми додаємо різний шум.

```
sdf_np = (-sdf_fn(pts_np)).astype(np.float32)
good = np.abs(sdf_np) < 5.0
pts_np, sdf_np = pts_np[good], sdf_np[good]
```

- Обчислюємо GT SDF з pysdf.
- **Захисний фільтр:** pysdf зрізка повертає смітєвий $\sim 10^{19}$. Будь-що з $|sdf| > 5$ — гарантовано сміття, бо діагональ нашого куба $\sim \sqrt{3} \approx 1.73$.

```
p = torch.from_numpy(pts_np).to(DEVICE); s = torch.from_numpy(sdf_np).to(DEVICE)
pred = model(p)
loss = (pred - s).abs().mean()
opt.zero_grad(set_to_none=True); loss.backward(); opt.step()
```

- **torch.from_numpy** — конвертація numpy → torch (zero-copy, ефективно).
- **pred = model(p)** — forward pass.
- **loss = ... abs().mean()** — L1-loss.
- **opt.zero_grad(set_to_none=True)** — обнуляє накопичені градієнти.
set_to_none=True — замість заповнення нулями, ставить **None** (трохи швидше).
- **loss.backward()** — обчислює градієнти автоматичним диференціюванням.
- **opt.step()** — оновлює ваги по Adam-правилу.

```
if (it + 1) % eval_every == 0 or it == n_iters - 1:
    with torch.no_grad(): pe = model(eval_pts_t).cpu().numpy()
    f1n = compute_f1(pe[:n_e], eval_gi_near)
    f1u = compute_f1(pe[n_e:], eval_gi_unif)
    history.append({'it': it+1, 'loss': float(loss), 'f1_near': f1n, 'f1_unif': f1u})
    if verbose:
        print(f' it={it+1:>4} loss={float(loss):.4f} f1_near={f1n:.3f} f1_unif={f1u:.3f}')
    if f1n >= early_stop_f1 and f1u >= early_stop_f1:
        if verbose: print(f' early stop at it={it+1}')
        break

return model, history
```

- **Кожні 250 ітерацій (або в останній)** — оцінюємо на eval-наборі.
- **with torch.no_grad():** — без накопичення градієнтів (ми не вчимо тут).
- Розраховуємо F1 окремо для near і unif.
- **Раннє зупинення:** якщо обидва $F1 \geq 0.96$, зупиняємось.

Клітинка 11: Тренувальний цикл

```
if RUN_TRAIN_ALL:
    log = []; grand_t0 = time.time()
    for p in mesh_files:
        mid = int(p.stem)
        out = MODELS_DIR / f'{mid}.npz'
        if out.exists():
            print(f'[{mid:>2}] cached - skipping'); continue
```

- Цикл по всіх 50 mesh-ах.

- **Idempotent**: якщо модель вже існує, пропускаємо. Зручно: можна перервати і продовжити з місця зупинки.

```
t0 = time.time()
model, history = train_one_mesh(mid, verbose=False)
model.export_npz(out)
size_kb = out.stat().st_size / 1024
last = history[-1] if history else {'f1_near': float('nan'), 'f1_unif': float('nan')}
print(f'[{mid:>2}] {time.time()-t0:5.1f}s  size={size_kb:6.1f} KB  '
      f'f1_near={last["f1_near"]:.3f}  f1_unif={last["f1_unif"]:.3f}  RSS={mem_mb():.0f}')
log.append({'id': mid, **last, 'size_kb': size_kb})
del model; gc.collect()
pd.DataFrame(log).to_csv(MODELS_DIR / 'train_log.csv', index=False)
print(f'\nTotal training time: {(time.time()-grand_t0)/60:.1f} min')
```

- Для кожного mesh: тренуємо, експортуємо, логуємо.
- `del model; gc.collect()` — звільняємо пам'ять після кожного mesh.
- `log.append({'id': mid, **last, 'size_kb': size_kb})` — `**last` розпаковує словник `last` як кейворд-аргументи (`f1_near`, `f1_unif`).
- В кінці пишемо CSV з логом усіх тренувань.

Клітинка 13: Оцінка

```
if RUN_EVAL_ALL:
    rows = []
    for p in mesh_files:
        mid = int(p.stem)
        npz = MODELS_DIR / f'{mid}.npz'
        if not npz.exists():
            print(f'[{mid:>2}] no model - skipping'); continue
        mesh = normalize_mesh(load_mesh(MESH_DIR / f'{mid}.obj'))
        sdf_fn = pysdf.SDF(mesh.vertices, mesh.faces)
        rng_e = np.random.default_rng(7777 + mid); n_e = 10_000
        base_e = trimesh.sample.sample_surface(mesh, n_e)[0]
        eval_pts = np.concatenate([
            (base_e + rng_e.normal(0, 1e-2, (n_e, 3))).astype(np.float32),
            rng_e.uniform(-1, 1, (n_e, 3)).astype(np.float32)])
        eval_sdf = (-sdf_fn(eval_pts)).astype(np.float32)
```

- Окрема оцінка з **новим RNG** (`7777 + mid`) — інші точки, ніж під час тренування.
- Це **справжня перевірка узагальнення**: ці точки модель ніколи не бачила.

```

model = MultiResHashSDF.from_npz(npz).to(DEVICE).eval()
with torch.no_grad():
    pe = model(torch.from_numpy(eval_pts).to(DEVICE)).cpu().numpy()
    f1n = compute_f1(pe[:n_e], eval_sdf[:n_e] < 0)
    f1u = compute_f1(pe[n_e:], eval_sdf[n_e:] < 0)
    size_kb = npz.stat().st_size / 1024
    rows.append({'id': mid, 'f1_near': f1n, 'f1_unif': f1u, 'size_kb': size_kb})
del mesh, model, sdf_fn; gc.collect()

```

- **.eval()** — переключає модель у режим оцінки (вимикає dropout, batch-norm-update, тощо). У нас немає dropout, але все одно гарна практика.

```

eval_df = pd.DataFrame(rows).sort_values('id').reset_index(drop=True)
print('\n=== Aggregate ===')
print(f'F1_near    mean={eval_df["f1_near"].mean():.3f}    min={eval_df["f1_near"].min():.3f}')
print(f'F1_unif    mean={eval_df["f1_unif"].mean():.3f}    min={eval_df["f1_unif"].min():.3f}')
print(f'size      mean={eval_df["size_kb"].mean():.1f} KB    max={eval_df["size_kb"].max():.1f}')
eval_df.to_csv(MODELS_DIR / 'eval.csv', index=False)
eval_df.head(10)

```

- Агрегована статистика — mean/min/max по всіх 50 mesh-ax.
- Зберігаємо `eval.csv`, який потім читається `retrain-cell`.

Клітинка 15: Перетренування важких mesh-ів

```

if RUN_RETRAIN_HARD:
    eval_df = pd.read_csv(MODELS_DIR / 'eval.csv')
    F1_NEAR_TGT, F1_UNIF_TGT = 0.92, 0.96
    hard = eval_df[(eval_df['f1_near'] < F1_NEAR_TGT) | (eval_df['f1_unif'] < F1_UNIF_TGT)]
    print(f'retraining {len(hard)} meshes: {hard["id"].tolist()}')

```

- Читаємо `eval.csv`.
- **Попір 0.92 / 0.96** — трохи вище за вимоги задачі (0.90 / 0.95). Запас на варіативність.
- **hard** — DataFrame з рядками, де якась з метрик нижче порогу.

```

big_cfg = HashConfig(n_levels=8, n_features=2, log2_table_size=14,
                    base_resolution=8, finest_resolution=128, hidden=16)

```

- **Збільшений варіант:** `log2_table_size=14` замість 13 = таблиці вдвічі більші.
- Розмір моделі — ~470 КБ замість ~236 КБ. Все одно у межах 1 МБ.

```

log = []; t0_all = time.time()
for mid in hard['id'].tolist():
    t0 = time.time()
    model, history = train_one_mesh(mid, n_iters=5000, cfg=big_cfg, verbose=False)
    out = MODELS_DIR / f'{mid}.npz'
    model.export_npz(out)
    ...

```

- **n_iters=5000** замість 2500 — довше тренування для важких випадків.
- **cfg=big_cfg** — передаємо більшу конфігурацію.
- Експорт **перезаписує** малу модель — це нормально, бо в .npz зберігається cfg.

Клітинка 17: Бенчмарк

```

if RUN_BENCH:
    from sdf_inference import SDFInference
    rng = np.random.default_rng(123)
    rows = []
    for p in mesh_files:
        mid = int(p.stem)
        npz = MODELS_DIR / f'{mid}.npz'
        if not npz.exists(): continue
        nb = SDFInference(npz); nb.warmup()

```

- **Імпорт всередині cell** — щоб не потрібно було імпортувати на верхньому рівні.
- **nb.warmup()** — triggers numba-компіляцію. Без цього перший виклик буде помилково повільним.

```

torch_m = MultiResHashSDF.from_npz(npz).eval()
pts = rng.uniform(-1, 1, (1000, 3)).astype(np.float32)
with torch.no_grad():
    tp = torch_m(torch.from_numpy(pts)).numpy()
    npp = nb(pts)
    max_diff = float(np.abs(tp - npp).max())

```

- **Тест паритету:** завантажуюємо ту саму модель в torch і в numba, прогонимо однакові точки.
- **max_diff** — максимальна абсолютна різниця. Має бути близько до нуля (ми хочемо $< 10^{-3}$).

```

ns_single = nb.bench_single_point(100_000)

bench = {}
for B in [1000, 10_000, 100_000]:
    bp = rng.uniform(-1, 1, (B, 3)).astype(np.float32)
    t = []
    for _ in range(20):
        t0 = time.perf_counter(); _ = nb(bp); t.append(time.perf_counter() - t0)
    bench[B] = float(np.median(t) * 1e3) # ms

```

- **Single-point benchmark** через compiled loop.
- **Batch benchmarks** для трьох розмірів. Беремо медіану з 20 спроб, щоб видалити ВИКИДИ.

```

rows.append({'id': mid,
            'parity_max_diff': max_diff,
            'size_kb': npz.stat().st_size / 1024,
            'ns_single': ns_single,
            'ms_1k': bench[1000],
            'ms_10k': bench[10_000],
            'ms_100k': bench[100_000]})
del torch_m, nb; gc.collect()

```

- Лог в DataFrame.

```

bench_df = pd.DataFrame(rows).sort_values('id').reset_index(drop=True)
print('\n=== Aggregate ===')
print(f'parity_max_diff    max={bench_df["parity_max_diff"].max():.2e}    target<1e-3')
print(f'size_kb           mean={bench_df["size_kb"].mean():.0f}    max={bench_df["size_kb"].max():.0f}')
print(f'ns_single         mean={bench_df["ns_single"].mean():.0f}    max={bench_df["ns_single"].max():.0f}')
print(f'ms_1k           (1 k pts) mean={bench_df["ms_1k"].mean():.3f}    max={bench_df["ms_1k"].max():.3f}')
print(f'ms_10k          (10k pts) mean={bench_df["ms_10k"].mean():.3f}    max={bench_df["ms_10k"].max():.3f}')
print(f'ms_100k         (100k pts) mean={bench_df["ms_100k"].mean():.3f}    max={bench_df["ms_100k"].max():.3f}')
bench_df.to_csv(MODELS_DIR / 'bench.csv', index=False)
bench_df.head(10)

```

- Агреговані метрики з вимогами task.
-

Підсумок

Файл	Призначення	Ключові частини
<code>sdf_model.py</code>	PyTorch-модель для тренування	<code>MultiResHashSDF.forward</code> (multi-res hash + MLP), <code>export_npz</code> / <code>from_npz</code> (серіалізація)
<code>sdf_inference.py</code>	Numba-кернел для inference	<code>_forward_one</code> (один запит), <code>_forward_batch</code> (з <code>prange</code>), <code>SDFInference</code> (зручний клас)
<code>train.ipynb</code>	Пайплайн повного циклу	EDA → візуалізація → <code>train_one_mesh</code> → loop → eval → retrain → benchmark

Уся ця система побудована за принципом **розділення відповідальностей**:

- **PyTorch** — для тренування (потрібен autograd, optimizer, тощо).
- **Numba** — для inference (нативна швидкість без Python-overhead).
- **.npz** — спільний формат між ними.

Тренування — ~1 година на M4 CPU для всіх 50 mesh-ів. Inference — менше за 1 мс на 1000 точок. Паритет між реалізаціями — точність $\sim 10^{-5}$. Все це працює в межах бюджету 1 МБ на mesh.